SQL ANALYTICS PROJECT · MYSQL 8

Compensation Analysis & Pay Equity

A MySQL analytics project on an employee compensation dataset — department inequality, hiring-sequence salary inversions, counterfactual workforce restructuring, benchmark instability under different ranking definitions, and a ranked list of five compensation-architecture risks. Every answer is built from chained CTEs, window functions and self-joins.

81	5	43.18%	0.2107
USD EMPLOYEES ANALYSED	DEPARTMENTS	SALARY INVERSION RATE	HIGHEST GINI · PRODUCT

DATABASE ENGINE

MySQL 8.0

InnoDB · CTEs, window functions and VAR_POP required

SCHEMA

compensation_analysis

1 source table · 5 analytical questions · 24 queries

SOURCE DATA

400 rows

4 currencies · 81 USD employees analysed

CONCEPTS COVERED

CTE · NTILE · RANK

Self-joins, Gini, median by ROW_NUMBER, conditional aggregation

WHAT THIS DOCUMENT CONTAINS

Every query of the original case-study script is reproduced with its executed result, a plain-language reading of what the result means, seven charts, the ranked five-risk executive summary the script was meant to produce, and a code-review section listing the defects and design observations found, with a correction for each.

CONTENTS

Table of Contents

01	**Project Overview** Scope, objectives and the analytical techniques the project demonstrates.	3
02	**Data Model & Scope** The single source table, its columns, the currency split and the USD analysis cohort.	5
03	**Database Setup & Validation** Steps 1–7: create the schema, import the CSV, check the data and compute total compensation.	6
04	**Inequality & Structural Pay Gaps** Question 1: department Gini, under-paid high performers, top-decile anomalies, like-for-like peer gaps.	8
05	**Hiring-Sequence Salary Inversions** Question 2: inversion rates by department, the most-involved employees, and segmentation.	14
06	**Counterfactual Workforce Restructuring** Question 3: effect of excluding L7 and L8, and single-employee removal ranking.	20
07	**Compensation Trajectory & Benchmark Instability** Question 4: three ranking definitions, rank movers and employees stable in the top 10%.	26
08	**Executive Recommendation** Question 5: the five ranked compensation-architecture risks with indicator, population and magnitude.	31
09	**Code Review & Corrections** Defects found while checking the script against its own requirements, with the fix for each.	36
10	**Results, Setup & Roadmap** How to run the project, object inventory and future enhancements.	39

SECTION

01 Project Overview

Scope, objectives and the analytical techniques the project demonstrates.

The **compensation_analysis** project takes one flat employee-compensation export (400 employees, five departments, four pay currencies) and answers the questions a compensation committee would ask: where is pay most unequal, who is paid below their peers despite strong performance, does a later hire out-earn an earlier one, how much of the picture depends on the most senior levels, and how fragile are pay rankings to the definition of the comparison group. The work is done entirely in SQL using total compensation defined as `base_salary + annual_bonus + equity_value`.

400	81	5	24
SOURCE ROWS	USD COHORT	DEPARTMENTS	QUERIES

Objectives

- **Measure inequality.** A pairwise Gini coefficient per department on total compensation, alongside each department's median.
- **Find structural pay gaps.** Employees below their department median despite an above-median rating; top-decile earners below their department-and-level median; spread inside identical peer cells.
- **Detect hiring-sequence inversions.** Pairs where the later hire earns more base salary than the earlier hire in the same department and job family, with rates, a department league table, the most-involved employees and segmentation by level, location and rating.
- **Run counterfactuals.** Recompute mean, median and inequality with L7 and L8 excluded, and rank single-employee removals by their effect on salary variance.
- **Test benchmark stability.** Rank everyone under three definitions (all, active only, active hired in the last five years) and isolate rank movers and stable top-10% earners.
- **Recommend.** Convert the measurements into five ranked architecture risks, each with an indicator, an affected population and a magnitude.

Analytical Modules

OUTPUT · MODULES OF THE ANALYSIS

Module	Question	What it answers
Inequality & pay gaps	Question 1	Department Gini and median; high performers below median; top-decile anomalies; peer-cell gaps.
Salary inversions	Question 2	Inversion rate per department; employees in the most inversions; breakdown by level, location, rating.
Counterfactuals	Question 3	Mean, median and Gini with L7/L8 removed; one-at-a-time removal ranked by variance change.
Benchmark instability	Question 4	Three rankings; rank improvement of 5+ places; employees in the top 10% under all three.
Executive summary	Question 5	Five ranked risks with indicator, population and magnitude.

Technology and Techniques

OUTPUT · TECHNIQUES USED

Area	Applied in this project
DDL / DML	CREATE DATABASE, USE, CSV import into one source table
CTE pipelines	WITH chains of up to four named steps feeding a final SELECT
Joins	Self-joins on department + job family for hiring-order pairs; department self-joins for the Gini numerator; LEFT JOIN of three rankings
Window functions	ROW_NUMBER() and COUNT() OVER for medians; RANK(); NTILE(10) for deciles; peer-cell COUNT/AVG/MIN/MAX OVER (PARTITION BY ...)
Aggregation	COUNT, SUM, AVG, MIN, MAX, VAR_POP, COUNT(DISTINCT), conditional SUM(CASE WHEN ...)
Medians in MySQL	No MEDIAN() exists: the middle one or two rows are picked with ROW_NUMBER and averaged
Inequality maths	Gini = $\Sigma\lvert x_i - x_j\rvert$ / $(2 \cdot n^2 \cdot \text{mean})$ computed from a department self-join
Null-safe maths	COALESCE on bonus and equity; NULLIF in every ratio denominator
Date logic	DATE_SUB(CURDATE(), INTERVAL 5 YEAR) for the recent-hire window

SECTION

02 Data Model & Scope

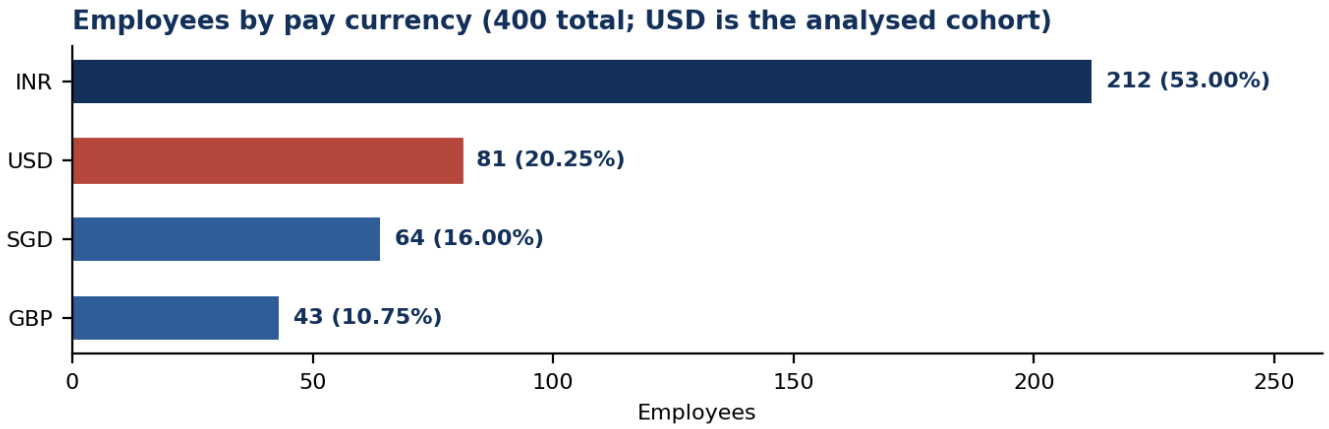
The single source table, its columns, the currency split and the USD analysis cohort.

The project uses a single denormalised table loaded from a CSV (imported by the Workbench wizard as `employee_compensation (1)`). There are no foreign keys: department, job family and location are free-text attributes of the employee row, and the manager link is a plain id column.

OUTPUT · COLUMNS OF THE SOURCE TABLE

Column	Meaning	Used for
employee_id	Employee key such as EMP0001	Identity, tie-breaks, pair construction
department	Analytics, Engineering, Operations, Product, Sales	Grouping for Gini, medians, inversions
job_level	L1 … L8	Peer groups; L7/L8 counterfactual
job_family	Product, Operations, Finance, Software, Data, People, Sales	Inversion pairs and peer cells
manager_id	Manager’s employee id	Not used
hire_date	Date of hire (dd-mm-yyyy in the export)	Hiring order; five-year window
employment_status	Active, On Leave, Exited	Ranking definitions
base_salary	Annual base pay in *currency*	Inversions, variance, medians
annual_bonus, equity_value	Variable pay (may be NULL)	Total compensation
performance_rating	1–5	High-performer test
promotion_date	Last promotion (often blank)	Not used
work_location	New York, London, Mumbai, Bengaluru, Singapore	Location segmentation
currency	USD, INR, GBP, SGD	Cohort filter

Currency Split and the USD Cohort



Salaries are stored in each employee’s own currency and there is no exchange-rate table, so figures in different currencies cannot be pooled. Every query therefore filters `WHERE currency = 'USD'`, which leaves **81 of 400 employees (20.25%)**. All numbers in this document describe that USD cohort, not the whole company. Within it: 72 Active, 3 On Leave and 6 Exited employees; 14 Analytics, 28 Engineering, 11 Operations, 13 Product and 15 Sales.

> **SCOPE NOTE**
> Conclusions about “the company” should be read as conclusions about the 81 USD-paid employees. Section 09 (Defect 6) explains why the other 319 employees are excluded and what would be needed to include them.

SECTION

03 Database Setup & Validation

Steps 1–7: create the schema, import the CSV, check the data and compute total compensation.

The statements below are reproduced as executed. The CSV import (Step 4 of the script) is performed in the MySQL Workbench Table Data Import Wizard, so it has no SQL text.

Step 1 — Create and select the schema

STEP 1 — SCHEMA CREATION

```
CREATE DATABASE compensation_analysis;
USE compensation_analysis;
SHOW TABLES;
-- Confirm the selected database
SELECT DATABASE();
```

Step 2 — Import the CSV

Import the file through *Schema → Tables → Table Data Import Wizard*. The wizard creates the table named `employee_compensation (1)` after the file name. Because the name contains a space and brackets it must be quoted with backticks in every query.

Step 3 — Check the data

STEP 3 — PREVIEW TEN ROWS

```
SELECT *
FROM `employee_compensation (1)`
LIMIT 10;
```

OUTPUT · FIRST ROWS OF THE IMPORTED TABLE (SELECTED COLUMNS)

employee_id	department	level	hire_date	status	base_salary	rating	work_location	currency
EMP0001	Engineering	L3	05-11-2021	Active	104000	2	New York	USD
EMP0002	Analytics	L3	08-12-2012	On Leave	106000	4	Bengaluru	INR
EMP0003	Product	L7	01-07-2020	Exited	160000	2	London	INR
EMP0004	Operations	L3	19-06-2013	Active	81000	4	Mumbai	GBP
EMP0005	Product	L3	20-10-2017	Active	99000	3	Mumbai	USD
EMP0006	Sales	L2	08-04-2021	Active	84000	5	Bengaluru	USD
EMP0007	Analytics	L4	24-08-2021	Active	108000	4	New York	SGD
EMP0008	Engineering	L4	26-01-2012	Active	136000	3	Mumbai	INR

Several rows pair a currency with a work location that does not match it (EMP0003 is based in London but paid in INR; EMP0004 in Mumbai but paid in GBP; EMP0007 in New York but paid in SGD). The currency column is therefore treated as the pay currency, independent of location.

Step 4 — Total employees and currency check

STEP 4 — HEADCOUNT BY CURRENCY

```
SELECT
  currency,
  COUNT(*) AS employee_count,
  COUNT(DISTINCT employee_id) AS unique_employees
FROM `employee_compensation (1)`
GROUP BY currency
ORDER BY currency;
```

OUTPUT · EMPLOYEES BY CURRENCY

currency	employee_count	unique_employees
GBP	43	43
INR	212	212
SGD	64	64
USD	81	81

Row count equals distinct employee count in every currency, so `employee_id` behaves as a key (no duplicate employees). The four counts add to 400.

Step 5 — Total compensation (USD)

Total compensation is base salary plus annual bonus plus equity value; `COALESCE(…,0)` stops a missing bonus or equity value from turning the whole sum into NULL.

STEP 5 — TOTAL COMPENSATION, USD EMPLOYEES

```
SELECT
  employee_id,
  department,
  job_level,
  job_family,
  work_location,
  base_salary,
  COALESCE(annual_bonus, 0) AS annual_bonus,
  COALESCE(equity_value, 0) AS equity_value,
  base_salary
    + COALESCE(annual_bonus, 0)
    + COALESCE(equity_value, 0) AS total_compensation,
  performance_rating,
  currency
FROM `employee_compensation (1)`
WHERE currency = 'USD'
ORDER BY total_compensation DESC;
```

OUTPUT · HIGHEST TOTAL COMPENSATION (FIRST 10 OF 81 ROWS)

employee_id	department	level	family	location	base	bonus	equity	total_comp	rating
EMP0252	Operations	L6	Operations	New York	160,000	46,868.61	74,245.00	281,113.61	3
EMP0211	Operations	L8	Finance	Bengaluru	195,000	27,469.43	38,995.32	261,464.75	2
EMP0323	Operations	L6	People	Mumbai	154,000	44,324.50	61,731.54	260,056.04	4
EMP0181	Engineering	L6	People	Singapore	159,000	19,608.64	72,513.69	251,122.33	4
EMP0269	Sales	L5	Operations	London	136,000	38,095.98	65,460.12	239,556.10	4
EMP0300	Product	L7	Product	Mumbai	174,000	9,545.13	53,479.28	237,024.41	3
EMP0265	Sales	L5	Product	Mumbai	175,000	27,241.62	28,635.35	230,876.97	4
EMP0059	Engineering	L6	Product	Mumbai	176,000	4,830.17	43,701.36	224,531.53	1
EMP0087	Engineering	L5	Product	New York	136,000	42,089.62	45,307.01	223,396.63	4
EMP0069	Engineering	L6	Data	New York	189,000	22,493.34	9,074.52	220,567.86	4

KEY INSIGHT

The top total-compensation list is dominated by Operations (three of the top four) and by equity: EMP0252, EMP0181 and EMP0323 each receive more than 60,000 USD in equity alone. Note that EMP0059 has the lowest possible performance rating (1) yet sits eighth by total compensation.

SECTION

04 Inequality & Structural Pay Gaps

Question 1: department Gini, under-paid high performers, top-decile anomalies, like-for-like peer gaps.

Business question. Calculate department-level compensation inequality using total compensation. Identify employees below their department median despite above-median performance; employees in the company's top 10% who are below their job-level/department median; and whether disparities persist among comparable job levels, job families and locations.

Question 1.1 — Department inequality (Gini) and median

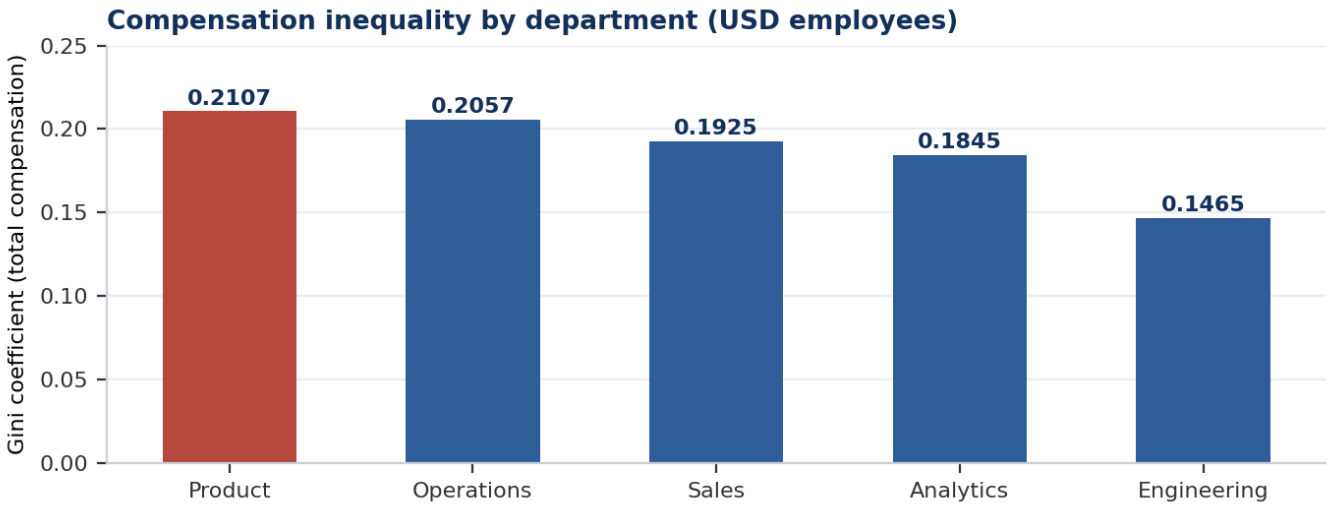
The Gini coefficient is computed from the full department self-join: the sum of absolute differences between every ordered pair, divided by $2 \cdot n^2 \cdot \text{mean}$. 0 means everyone is paid the same; values near 1 mean one person receives nearly everything. The median is the average of the one or two middle rows picked by `ROW_NUMBER`.

QUESTION 1.1 — DEPARTMENT GINI AND MEDIAN

```
WITH comp AS (
  SELECT
    employee_id,
    department,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_comp
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
gini AS (
  SELECT
    a.department,
    SUM(ABS(a.total_comp - b.total_comp))
      / NULLIF( 2 * POW(COUNT(DISTINCT a.employee_id), 2)
                  * AVG(a.total_comp),
                0
      ) AS gini_coefficient
  FROM comp a
  JOIN comp b
    ON a.department = b.department
  GROUP BY a.department
),
ranked AS (
  SELECT
    department,
    total_comp,
    ROW_NUMBER() OVER (
      PARTITION BY department
      ORDER BY total_comp
    ) AS rn,
    COUNT(*) OVER (
      PARTITION BY department
    ) AS cnt
  FROM comp
),
medians AS (
  SELECT
    department,
    AVG(total_comp) AS median_compensation
  FROM ranked
  WHERE rn IN (
    FLOOR((cnt + 1) / 2),
    CEIL((cnt + 1) / 2)
  )
  GROUP BY department
)
SELECT
  g.department,
  m.median_compensation,
  ROUND(g.gini_coefficient, 4) AS gini_coefficient
FROM gini g
JOIN medians m
  ON g.department = m.department
ORDER BY g.gini_coefficient DESC;
```

OUTPUT · DEPARTMENT MEDIAN AND GINI

department	median_compensation	gini_coefficient
Product	119,418.25	0.2107
Operations	175,642.30	0.2057
Sales	150,499.50	0.1925
Analytics	151,338.915	0.1845
Engineering	164,020.73	0.1465



KEY INSIGHT

Product is the most unequal department (Gini 0.2107) and has the lowest median (119,418.25). Its Gini is 43.8% higher than Engineering's (0.1465), the most equal. Operations pairs a high median (175,642.30) with the second-highest Gini, so it pays well but unevenly. All five values are moderate in absolute terms (all below 0.25); the finding is about relative dispersion between departments, not extreme inequality.

Question 1.2 — High performance, below-median compensation

An employee qualifies when their total compensation is below the department median **and** their rating is above the department's median rating. The rating median is calculated with the same ROW_NUMBER technique as the pay median.

QUESTION 1.2 — HIGH PERFORMERS BELOW MEDIAN PAY

```
WITH comp AS (
  SELECT
    employee_id,
    department, job_level,
    job_family,
    work_location,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation,
    performance_rating
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
department_medians AS (
  SELECT
    department,
    AVG(total_compensation) AS median_compensation
  FROM (
    SELECT
      department,
      total_compensation,
      ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY total_compensation
      ) AS rn,
      COUNT(*) OVER (
        PARTITION BY department
      ) AS cnt
```

QUESTION 1.2 — HIGH PERFORMERS BELOW MEDIAN PAY

```
    FROM comp
  ) ranked
  WHERE rn IN (
    FLOOR((cnt + 1) / 2),
    CEIL((cnt + 1) / 2)
  ) GROUP BY department
),
performance_medians AS (
  SELECT
    department,
    AVG(performance_rating) AS median_rating
  FROM (
    SELECT
      department,
      performance_rating,
      ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY performance_rating
      ) AS rn,
      COUNT(*) OVER (
        PARTITION BY department
      ) AS cnt
    FROM comp
  ) ranked
  WHERE rn IN (
    FLOOR((cnt + 1) / 2),
    CEIL((cnt + 1) / 2)
  )
  GROUP BY department
)
SELECT
  c.employee_id,
  c.department,
  c.job_level,
  c.job_family,
  c.work_location,
  c.total_compensation,
  c.performance_rating,
  dm.median_compensation,
  pm.median_rating
FROM comp c
JOIN department_medians dm
  ON c.department = dm.department
JOIN performance_medians pm
  ON c.department = pm.department
WHERE c.total_compensation < dm.median_compensation
  AND c.performance_rating > pm.median_rating
ORDER BY c.department, c.total_compensation;
```

OUTPUT · EMPLOYEES RATED ABOVE THEIR DEPARTMENT MEDIAN BUT PAID BELOW IT

employee_id	department	level	family	location	total_comp	rating	dept_median	median_rating
EMP0078	Analytics	L3	People	New York	99,553.46	5	151,338.915	4
EMP0030	Engineering	L2	Data	London	75,888.60	5	164,020.73	4
EMP0029	Engineering	L2	Product	Mumbai	89,243.37	5	164,020.73	4
EMP0184	Engineering	L3	Data	Singapore	145,603.29	5	164,020.73	4
EMP0288	Engineering	L4	People	Bengaluru	148,609.20	5	164,020.73	4
EMP0262	Product	L3	Operations	London	114,503.77	4	119,418.25	3
EMP0006	Sales	L2	Software	Bengaluru	100,903.77	5	150,499.50	4

KEY INSIGHT

Seven employees (8.6% of the cohort) match, four of them in Engineering. The largest shortfall is EMP0030: 75,888.60 against a department median of 164,020.73, i.e. 88,132.13 or 53.7% below, despite a rating of 5. Part of that gap is explained by level (L2 against a median that mixes all levels), which is why Question 1.3 and Question 1.4 compare like with like.

Question 1.3 — Top-decile earners below their peer median

Employees in the first `NTILE(10)` bucket of company-wide total compensation (the top 10%, nine of 81 people) are compared with the median of their own department and job level.

QUESTION 1.3 — TOP 10% BELOW DEPARTMENT/LEVEL MEDIAN

```
WITH comp AS (
  SELECT
    employee_id,
    department,
    job_level,
    job_family,
    work_location,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
ranked AS (
  SELECT
    *,
    NTILE(10) OVER (
      ORDER BY total_compensation DESC
    ) AS pay_decile
  FROM comp
),
peer_medians AS (
  SELECT
    department,
    job_level,
    AVG(total_compensation) AS peer_median
  FROM (
    SELECT
      department,
      job_level,
      total_compensation,
      ROW_NUMBER() OVER (
        PARTITION BY department, job_level
        ORDER BY total_compensation
      ) AS rn,
      COUNT(*) OVER (
        PARTITION BY department, job_level
      ) AS cnt
    FROM comp
  ) x
  WHERE rn IN (
    FLOOR((cnt + 1) / 2),
    CEIL((cnt + 1) / 2)
  )
  GROUP BY department, job_level
)
SELECT
  r.employee_id,
  r.department,
  r.job_level,
  r.job_family,
  r.work_location,
  r.total_compensation,
  p.peer_median
FROM ranked r
JOIN peer_medians p
  ON r.department = p.department
 AND r.job_level = p.job_level
WHERE r.pay_decile = 1
  AND r.total_compensation < p.peer_median
ORDER BY r.total_compensation DESC;
```

OUTPUT · TOP-DECILE EMPLOYEES BELOW THEIR DEPARTMENT-LEVEL MEDIAN

employee_id	department	level	family	location	total_comp	peer_median
EMP0323	Operations	L6	People	Mumbai	260,056.04	270,584.825

KEY INSIGHT

Only one of the nine top-decile employees qualifies, and the margin is small: EMP0323 earns 10,528.79 (3.9%) below the peer median. The Operations L6 peer group contains only two people (EMP0252 at 281,113.61 and EMP0323), so the “median” is simply their midpoint — the result is a statement about a two-person cell, not evidence of a broader pattern.

Question 1.4 — Disparities among comparable employees

Peers are defined as employees sharing **department, job level, job family and work location**. Window functions attach the peer count, average, minimum and maximum to every row; only cells with at least two people and a non-zero spread are returned. (The script labels this block “Task 4”; see Section 09.)

QUESTION 1.4 — PEER-CELL COMPENSATION GAPS

```
WITH comp AS (
  SELECT
    employee_id,
    department,
    job_level, job_family,
    work_location,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
peer_comparison AS (
  SELECT
    employee_id,
    department,
    job_level,
    job_family,
    work_location,
    total_compensation,
    COUNT(*) OVER (
      PARTITION BY department, job_level, job_family, work_location
    ) AS peer_count,
    AVG(total_compensation) OVER (
      PARTITION BY department, job_level, job_family, work_location
    ) AS peer_avg_compensation,
    MIN(total_compensation) OVER (
      PARTITION BY department, job_level, job_family, work_location
    ) AS peer_min_compensation,
    MAX(total_compensation) OVER (
      PARTITION BY department, job_level, job_family, work_location
    ) AS peer_max_compensation
  FROM comp
)
SELECT
  employee_id,
  department,
  job_level,
  job_family,
  work_location,
  total_compensation,
  peer_count,
  ROUND(peer_avg_compensation, 2) AS peer_avg_compensation,
  ROUND(peer_min_compensation, 2) AS peer_min_compensation,
  ROUND(peer_max_compensation, 2) AS peer_max_compensation,
  ROUND(
    peer_max_compensation - peer_min_compensation, 2
  ) AS compensation_gap
FROM peer_comparison
WHERE peer_count >= 2
  AND peer_max_compensation > peer_min_compensation
ORDER BY compensation_gap DESC, department, job_level;
```

OUTPUT · PEER CELLS WITH A COMPENSATION GAP

employee_id	department	level	family	location	total_comp	peers	peer_avg	peer_min	peer_max	gap
EMP0032	Sales	L5	Operations	London	147,731.69	2	193,643.90	147,731.69	239,556.10	91,824.41
EMP0269	Sales	L5	Operations	London	239,556.10	2	193,643.90	147,731.69	239,556.10	91,824.41
EMP0147	Analytics	L4	Operations	New York	135,592.42	2	141,438.62	135,592.42	147,284.81	11,692.39
EMP0126	Analytics	L4	Operations	New York	147,284.81	2	141,438.62	135,592.42	147,284.81	11,692.39
EMP0012	Engineering	L5	Operations	Bengaluru	166,036.10	2	161,510.34	156,984.57	166,036.10	9,051.53
EMP0313	Engineering	L5	Operations	Bengaluru	156,984.57	2	161,510.34	156,984.57	166,036.10	9,051.53

KEY INSIGHT

Disparities do persist inside identical cells, but the evidence is thin: only three cells (six employees, 7.4% of the cohort) have two peers. The widest is Sales L5 Operations London, where one employee earns 62.2% more than the other (239,556.10 against 147,731.69, a gap of 91,824.41). The other two gaps are 8.6% (Analytics) and 5.8% (Engineering). Every cell in this table has exactly two people, so no cell is large enough to estimate a spread.

SECTION

05 Hiring-Sequence Salary Inversions

Question 2: inversion rates by department, the most-involved employees, and segmentation.

Business question. Identify pairs where a later-hired employee earns more base salary than an earlier-hired employee in the same department and job family. Calculate inversion rates, rank departments, identify the employees involved in the most inversions, and segment by job level, location and performance rating.

A **pair** is formed by self-joining the table on department and job family with `e2.hire_date > e1.hire_date`; an **inversion** is a pair in which `e2.base_salary > e1.base_salary`. The inversion rate is inverted pairs divided by comparable pairs.

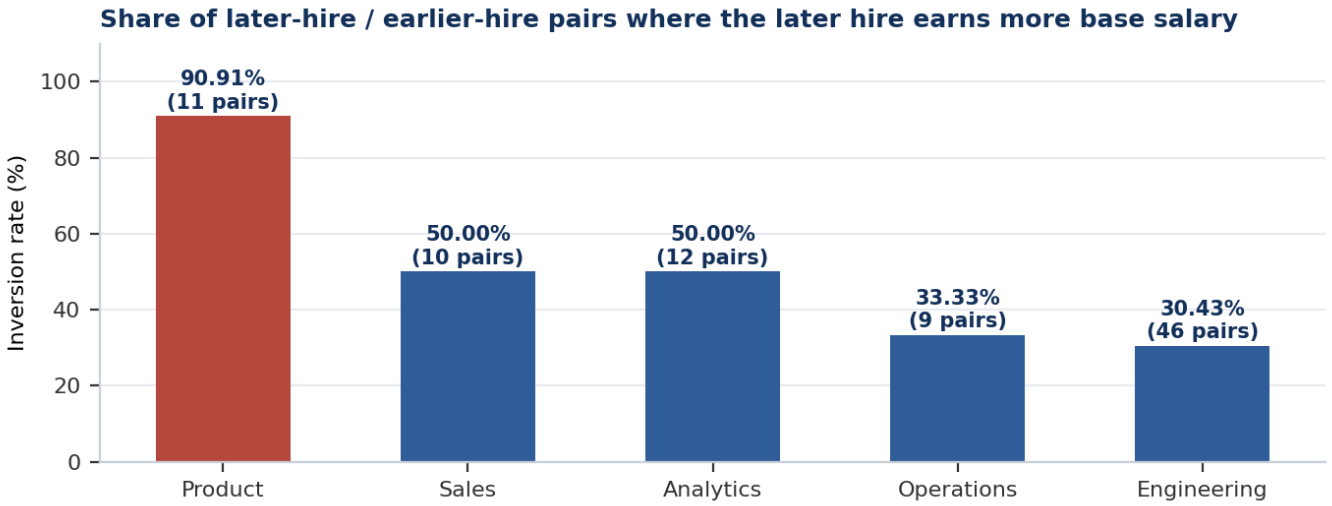
Step 1 — Department-level inversion rates and ranking

QUESTION 2, STEP 1 — INVERSION RATE BY DEPARTMENT

```
WITH inversion_summary AS (
  SELECT
    e1.department,
    COUNT(*) AS comparable_pairs,
    SUM(
      CASE
        WHEN e2.base_salary > e1.base_salary THEN 1
        ELSE 0
      END
    ) AS inverted_pairs,
    ROUND(
      100.0 * SUM(
        CASE
          WHEN e2.base_salary > e1.base_salary THEN 1 ELSE 0
        END
      ) / NULLIF(COUNT(*), 0),
      2
    ) AS inversion_rate_pct
  FROM `employee_compensation (1)` e1
  JOIN `employee_compensation (1)` e2
    ON e2.department = e1.department
   AND e2.job_family = e1.job_family
   AND e2.currency = e1.currency
   AND e2.hire_date > e1.hire_date
  WHERE e1.currency = 'USD'
    AND e2.currency = 'USD'
    AND e1.hire_date IS NOT NULL
    AND e2.hire_date IS NOT NULL
    AND e1.base_salary IS NOT NULL
    AND e2.base_salary IS NOT NULL
  GROUP BY e1.department
),
ranked_departments AS (
  SELECT
    *,
    DENSE_RANK() OVER (
      ORDER BY inversion_rate_pct DESC
    ) AS department_rank
  FROM inversion_summary
)
SELECT *
FROM ranked_departments
ORDER BY department_rank;
```

OUTPUT · INVERSION RATE AND RANK BY DEPARTMENT

department	comparable_pairs	inverted_pairs	inversion_rate_pct	department_rank
Product	11	10	90.91	1
Sales	10	5	50.00	2
Analytics	12	6	50.00	2
Operations	9	3	33.33	3
Engineering	46	14	30.43	4



KEY INSIGHT

Across the five departments there are 88 comparable pairs and 38 inversions, an overall inversion rate of **43.18%**. Product is the outlier: 10 of its 11 pairs are inverted (90.91%). Engineering has by far the most pairs (46) but the lowest rate (30.43%). Sales and Analytics tie for second, which is why DENSE_RANK leaves no gap before rank 3.

Step 2 — Employees involved in the most inversions

Each inverted pair is unpivoted with `UNION ALL` so that the later hire (who is paid more) and the earlier hire (who is paid less) are each credited with one inversion.

QUESTION 2, STEP 2 — EMPLOYEES IN THE MOST INVERSIONS

```
WITH inversion_pairs AS (
  SELECT
    e1.employee_id AS earlier_employee_id,
    e2.employee_id AS later_employee_id,
    e1.department,
    e1.job_family,
    e1.job_level AS earlier_job_level,
    e2.job_level AS later_job_level,
    e1.work_location AS earlier_location,
    e2.work_location AS later_location,
    e1.performance_rating AS earlier_performance,
    e2.performance_rating AS later_performance
  FROM `employee_compensation (1)` e1
  JOIN `employee_compensation (1)` e2
    ON e2.department = e1.department
   AND e2.job_family = e1.job_family
   AND e2.currency = e1.currency
   AND e2.hire_date > e1.hire_date
  WHERE e1.currency = 'USD'
    AND e2.currency = 'USD'
    AND e1.hire_date IS NOT NULL
    AND e2.hire_date IS NOT NULL
    AND e1.base_salary IS NOT NULL AND e2.base_salary IS NOT NULL
    AND e2.base_salary > e1.base_salary
),
employee_counts AS (
  SELECT
    employee_id,
    department,
    SUM(later_inversions) AS later_hire_inversions,
    SUM(earlier_inversions) AS earlier_hire_inversions
  FROM (
    SELECT
      later_employee_id AS employee_id,
      department,
      1 AS later_inversions,
      0 AS earlier_inversions
    FROM inversion_pairs
    UNION ALL
    SELECT
      earlier_employee_id AS employee_id,
      department,
      0 AS later_inversions,
      1 AS earlier_inversions
    FROM inversion_pairs
```

QUESTION 2, STEP 2 — EMPLOYEES IN THE MOST INVERSIONS

```
  ) x
  GROUP BY employee_id, department
)
SELECT
  employee_id,
  department,
  later_hire_inversions,
  earlier_hire_inversions,
  later_hire_inversions + earlier_hire_inversions
    AS total_inversions
FROM employee_counts
ORDER BY total_inversions DESC, employee_id
LIMIT 20;
```

OUTPUT · TOP EMPLOYEES BY TOTAL INVERSIONS (FIRST 12 OF 20)

employee_id	department	later_hire_inversions	earlier_hire_inversions	total_inversions
EMP0030	Engineering	0	4	4
EMP0069	Engineering	3	0	3
EMP0148	Product	2	1	3
EMP0181	Engineering	3	0	3
EMP0184	Engineering	1	2	3
EMP0204	Product	0	3	3
EMP0207	Product	1	2	3
EMP0288	Engineering	0	3	3
EMP0300	Product	3	0	3
EMP0094	Analytics	2	0	2
EMP0107	Engineering	1	1	2
EMP0124	Engineering	1	1	2

KEY INSIGHT

EMP0030 is the most-affected earlier hire: four later colleagues in the same department and family earn more base salary than they do. The three employees who each out-earn three earlier hires — EMP0300 (Product), EMP0181 (Engineering) and EMP0069 (Engineering) — are also in the top ten by total compensation (ranks 6, 4 and 10 in Section 03), which links the inversion problem to the highest-paid, more recently hired people.

Step 3 — Inversions by job level

QUESTION 2, STEP 3 — BY JOB LEVEL

```
SELECT
  e1.department,
  e1.job_family,
  e1.job_level AS earlier_job_level,
  e2.job_level AS later_job_level,
  COUNT(*) AS comparable_pairs,
  SUM(
    CASE
      WHEN e2.base_salary > e1.base_salary THEN 1
      ELSE 0
    END
  ) AS inverted_pairs,
  ROUND(
    100.0 * SUM(
      CASE
        WHEN e2.base_salary > e1.base_salary THEN 1
        ELSE 0
      END
    ) / NULLIF(COUNT(*), 0),
    2
  ) AS inversion_rate_pct
FROM `employee_compensation (1)` e1
JOIN `employee_compensation (1)` e2
  ON e1.department = e2.department
 AND e1.job_family = e2.job_family
 AND e1.currency = e2.currency
 AND e2.hire_date > e1.hire_date
WHERE e1.currency = 'USD'
  AND e1.hire_date IS NOT NULL
  AND e2.hire_date IS NOT NULL
  AND e1.base_salary IS NOT NULL
  AND e2.base_salary IS NOT NULL
GROUP BY
  e1.department,
  e1.job_family,
  e1.job_level,
  e2.job_level
ORDER BY
  inversion_rate_pct DESC,
  comparable_pairs DESC;
```

OUTPUT · INVERSION RATE BY EARLIER AND LATER JOB LEVEL (FIRST 10 ROWS)

department	job_family	earlier_level	later_level	comparable_pairs	inverted_pairs	inversion_rate_pct
Engineering	Data	L3	L6	2	2	100
Product	Product	L2	L3	2	2	100
Engineering	People	L5	L6	2	2	100
Engineering	People	L4	L5	2	2	100
Product	Product	L2	L7	2	2	100
Analytics	People	L3	L6	2	2	100
Engineering	Data	L2	L3	2	2	100
Operations	People	L6	L5	1	1	100
Product	Operations	L3	L3	1	1	100
Product	Finance	L1	L3	1	1	100

Step 4 — Inversions by work location

QUESTION 2, STEP 4 — BY WORK LOCATION

```
SELECT
  e1.department,
  e1.job_family,
  e1.work_location AS earlier_location,
  e2.work_location AS later_location,
  COUNT(*) AS comparable_pairs,
  SUM(
    CASE
      WHEN e2.base_salary > e1.base_salary THEN 1
      ELSE 0
    END
  ) AS inverted_pairs,
  ROUND(
    100.0 * SUM(
      CASE
        WHEN e2.base_salary > e1.base_salary THEN 1
        ELSE 0
      END
    ) / NULLIF(COUNT(*), 0),
    2
  ) AS inversion_rate_pct
FROM `employee_compensation (1)` e1
JOIN `employee_compensation (1)` e2
  ON e1.department = e2.department
 AND e1.job_family = e2.job_family
 AND e1.currency = e2.currency
 AND e2.hire_date > e1.hire_date
WHERE e1.currency = 'USD'
  AND e1.hire_date IS NOT NULL
  AND e2.hire_date IS NOT NULL
  AND e1.base_salary IS NOT NULL
  AND e2.base_salary IS NOT NULL
GROUP BY
  e1.department,
  e1.job_family,
  e1.work_location,
  e2.work_location
ORDER BY
  inversion_rate_pct DESC,
  comparable_pairs DESC;
```

OUTPUT · INVERSION RATE BY EARLIER AND LATER LOCATION (FIRST 10 ROWS)

department	job_family	earlier_location	later_location	comparable_pairs	inverted_pairs	inversion_rate_pct
Engineering	People	Bengaluru	Singapore	2	2	100
Sales	Operations	London	London	1	1	100
Engineering	Data	London	Bengaluru	1	1	100
Product	Finance	New York	Mumbai	1	1	100
Sales	Software	Singapore	Bengaluru	1	1	100
Product	Operations	London	New York	1	1	100
Product	Operations	London	Mumbai	1	1	100
Engineering	Data	London	London	1	1	100
Engineering	Data	Singapore	London	1	1	100
Engineering	People	Bengaluru	Bengaluru	1	1	100

Step 5 — Inversions by performance rating

QUESTION 2, STEP 5 — BY PERFORMANCE RATING

```
SELECT
  e1.department,
  e1.job_family,
  e1.performance_rating AS earlier_performance_rating,
  e2.performance_rating AS later_performance_rating,
  COUNT(*) AS comparable_pairs,
  SUM(
    CASE
      WHEN e2.base_salary > e1.base_salary THEN 1
      ELSE 0
    END
  ) AS inverted_pairs,
  ROUND(
    100.0 * SUM(
      CASE
        WHEN e2.base_salary > e1.base_salary THEN 1
        ELSE 0
      END
    ) / NULLIF(COUNT(*), 0),
    2
  ) AS inversion_rate_pct
FROM `employee_compensation (1)` e1
JOIN `employee_compensation (1)` e2
  ON e1.department = e2.department
 AND e1.job_family = e2.job_family
 AND e1.currency = e2.currency
 AND e2.hire_date > e1.hire_date
WHERE e1.currency = 'USD'
  AND e1.hire_date IS NOT NULL
  AND e2.hire_date IS NOT NULL
  AND e1.base_salary IS NOT NULL
  AND e2.base_salary IS NOT NULL
  AND e1.performance_rating IS NOT NULL
  AND e2.performance_rating IS NOT NULL
GROUP BY
  e1.department,
  e1.job_family,
  e1.performance_rating,
  e2.performance_rating
ORDER BY
  inversion_rate_pct DESC,
  comparable_pairs DESC;
```

OUTPUT · INVERSION RATE BY EARLIER AND LATER RATING (FIRST 10 ROWS, PLUS THE LAST ROW)

department	job_family	earlier_rating	later_rating	comparable_pairs	inverted_pairs	inversion_rate_pct
Engineering	Data	5	4	3	3	100
Engineering	Data	5	2	2	2	100
Engineering	People	5	4	2	2	100
Product	Product	4	2	1	1	100
Analytics	Sales	4	5	1	1	100
Analytics	Sales	4	4	1	1	100
Product	Finance	3	3	1	1	100
Sales	Software	4	5	1	1	100
Analytics	Sales	5	4	1	1	100
Sales	Finance	3	3	1	1	100
…	…	…	…	…	…	…
Product	Operations	4	4	3	2	66.67

KEY INSIGHT

Segmenting by level, location or rating splits 88 pairs into so many cells that almost every cell holds one or two pairs and shows a 100% inversion rate; the one exception shown is Product/Operations at 66.67% (2 of 3). The first four rows of the rating table are the most informative: an earlier hire rated 5 is out-paid by later hires rated 4 or 2. Segments this small are descriptive, not statistically meaningful; see Defect 5 and Observation 13 in Section 09.

SECTION

06 Counterfactual Workforce Restructuring

Question 3: effect of excluding L7 and L8, and single-employee removal ranking.

Business question. For each department, calculate how mean salary, median salary and total-compensation inequality change when employees at levels L7 and L8 are excluded. Separately, rank individual employee removals by their marginal effect on department-level salary variance, with a minimum department-size requirement.

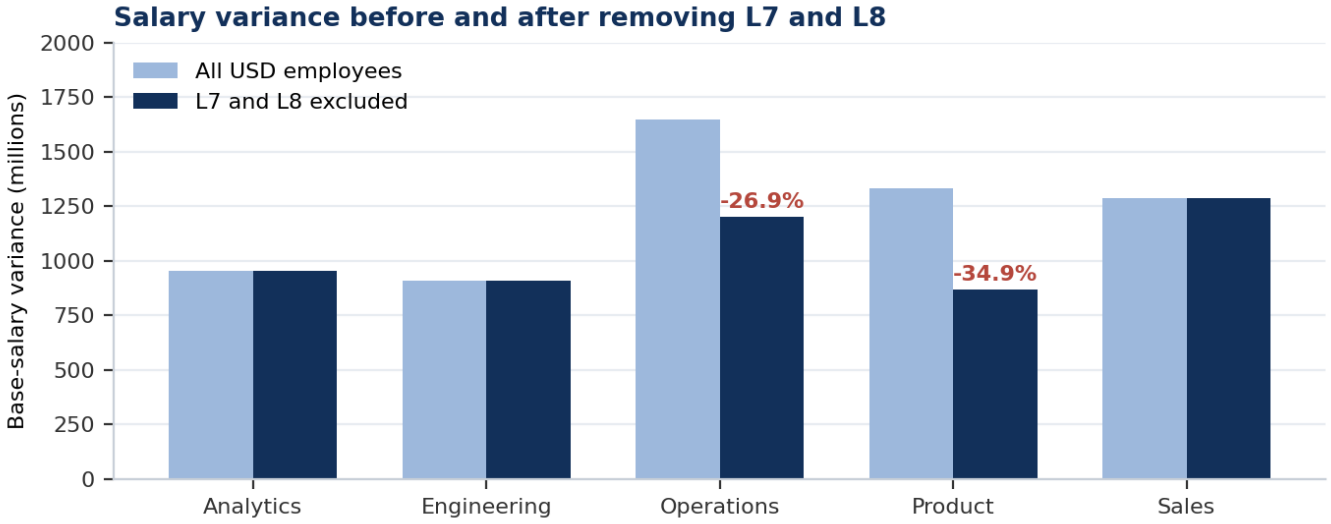
Step 1 — Mean salary and variance, before and after

QUESTION 3, STEP 1 — MEAN AND VARIANCE WITH AND WITHOUT L7/L8

```
WITH department_metrics AS (
  SELECT
    department,
    COUNT(base_salary) AS original_employee_count,
    AVG(base_salary) AS original_mean_salary,
    VAR_POP(base_salary) AS original_salary_variance,
    COUNT(
      CASE
        WHEN job_level NOT IN ('L7', 'L8')
        THEN base_salary
      END
    ) AS revised_employee_count,
    AVG(
      CASE
        WHEN job_level NOT IN ('L7', 'L8')
        THEN base_salary
      END
    ) AS revised_mean_salary,
    VAR_POP(
      CASE
        WHEN job_level NOT IN ('L7', 'L8')
        THEN base_salary
      END
    ) AS revised_salary_variance
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
  GROUP BY department
)
SELECT
  department,
  original_employee_count,
  revised_employee_count,
  ROUND(original_mean_salary, 2)
    AS original_mean_salary,
  ROUND(revised_mean_salary, 2)
    AS revised_mean_salary,
  ROUND(
    revised_mean_salary - original_mean_salary, 2
  ) AS mean_salary_change,
  ROUND(original_salary_variance, 2)
    AS original_salary_variance,
  ROUND(revised_salary_variance, 2)
    AS revised_salary_variance,
  ROUND(
    revised_salary_variance - original_salary_variance, 2
  ) AS variance_change
FROM department_metrics
ORDER BY department;
```

OUTPUT · BASE SALARY: ORIGINAL VS L7/L8 EXCLUDED

department	n_orig	n_rev	mean_orig	mean_rev	mean_change	var_orig	var_rev	var_change
Analytics	14	14	95,714.29	95,714.29	0	951,061,224.5	951,061,224.5	0
Engineering	28	28	117,821.43	117,821.43	0	906,146,683.7	906,146,683.7	0
Operations	11	10	120,727.27	113,300.00	−7,427.27	1,644,925,620	1,202,610,000	−442,315,619.8
Product	13	12	94,076.92	87,416.67	−6,660.26	1,333,147,929	867,576,388.9	−465,571,540.1
Sales	15	15	114,133.33	114,133.33	0	1,285,582,222	1,285,582,222	0



KEY INSIGHT

Only two employees in the USD cohort sit at L7 or L8: one in Operations (EMP0211, L8) and one in Product (EMP0300, L7). Removing them changes nothing in the other three departments. In Operations the mean base salary falls 6.15% and variance 26.89%; in Product the mean falls 7.08% and variance 34.92%.

Step 2 — Median salary, before and after

Medians use the same middle-row technique. The original median is picked from one ranking of all USD employees; the revised median from a second ranking that excludes L7 and L8.

QUESTION 3, STEP 2 — MEDIAN BASE SALARY

```
WITH ranked_salaries AS (
  SELECT
    department,
    base_salary,
    job_level,
    ROW_NUMBER() OVER (
      PARTITION BY department
      ORDER BY base_salary
    ) AS original_rank,
    COUNT(*) OVER (
      PARTITION BY department
    ) AS original_count,
    ROW_NUMBER() OVER (
      PARTITION BY department,
        CASE
          WHEN job_level NOT IN ('L7', 'L8') THEN 1
        END
      ORDER BY base_salary
    ) AS revised_rank
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
median_values AS (
  SELECT
    department,
```

QUESTION 3, STEP 2 — MEDIAN BASE SALARY

```
    AVG(
      CASE
        WHEN original_rank IN (
          FLOOR((original_count + 1) / 2),
          FLOOR((original_count + 2) / 2)
        )
        THEN base_salary
      END
    ) AS original_median_salary,
    AVG(
      CASE
        WHEN job_level NOT IN ('L7', 'L8')
        THEN base_salary
      END
    ) AS revised_mean_check
  FROM ranked_salaries
  GROUP BY department
),
revised_ranked AS (
  SELECT
    department,
    base_salary,
    ROW_NUMBER() OVER (
      PARTITION BY department
      ORDER BY base_salary
    ) AS salary_rank,
    COUNT(*) OVER (
      PARTITION BY department
    ) AS total_count
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
    AND job_level NOT IN ('L7', 'L8')
),
revised_medians AS (
  SELECT
    department,
    AVG(
      CASE
        WHEN salary_rank IN (
          FLOOR((total_count + 1) / 2),
          FLOOR((total_count + 2) / 2)
        )
        THEN base_salary
      END
    ) AS revised_median_salary
  FROM revised_ranked
  GROUP BY department
)
SELECT
  m.department,
  ROUND(m.original_median_salary, 2) AS original_median_salary,
  ROUND(r.revised_median_salary, 2) AS revised_median_salary,
  ROUND(
    r.revised_median_salary - m.original_median_salary, 2
  ) AS median_salary_change
FROM median_values m
LEFT JOIN revised_medians r
  ON m.department = r.department
ORDER BY m.department;
```

OUTPUT · MEDIAN BASE SALARY: ORIGINAL VS L7/L8 EXCLUDED

department	original_median_salary	revised_median_salary	median_salary_change
Analytics	98,500	98,500	0
Engineering	117,000	117,000	0
Operations	117,000	108,500	−8,500
Product	90,000	88,500	−1,500
Sales	126,000	126,000	0

Step 3 — Total-compensation inequality (Gini), before and after

QUESTION 3, STEP 3 — GINI UNDER BOTH SCENARIOS

```
WITH employee_comp AS (
  SELECT
    employee_id,
    department,
    job_level,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
scenarios AS (
  SELECT
    employee_id,
    department,
    'Original' AS scenario,
    total_compensation
  FROM employee_comp
  UNION ALL
  SELECT employee_id,
    department,
    'L7_L8_Excluded' AS scenario,
    total_compensation
  FROM employee_comp
  WHERE job_level NOT IN ('L7', 'L8')
),
department_stats AS (
  SELECT
    department,
    scenario,
    COUNT(*) AS employee_count,
    AVG(total_compensation) AS mean_compensation
  FROM scenarios
  GROUP BY department, scenario
),
pair_differences AS (
  SELECT
    a.department,
    a.scenario,
    SUM(ABS(a.total_compensation - b.total_compensation))
      AS difference_sum
  FROM scenarios a
  JOIN scenarios b
    ON a.department = b.department
   AND a.scenario = b.scenario
  GROUP BY a.department, a.scenario
)
SELECT
  d.department,
  d.scenario,
  d.employee_count,
  ROUND(d.mean_compensation, 2) AS mean_compensation,
  ROUND(
    p.difference_sum /
    (2 * d.employee_count * d.employee_count * d.mean_compensation),
    4
  ) AS gini_coefficient
FROM department_stats d
JOIN pair_differences p
  ON d.department = p.department
 AND d.scenario = p.scenario
ORDER BY d.department, d.scenario;
```

OUTPUT · MEAN TOTAL COMPENSATION AND GINI BY SCENARIO

department	scenario	employee_count	mean_compensation	gini_coefficient
Analytics	L7_L8_Excluded	14	144,227.64	0.1845
Analytics	Original	14	144,227.64	0.1845
Engineering	L7_L8_Excluded	28	165,053.16	0.1465
Engineering	Original	28	165,053.16	0.1465
Operations	L7_L8_Excluded	10	169,889.99	0.2049
Operations	Original	11	178,214.97	0.2057
Product	L7_L8_Excluded	12	118,985.69	0.1835
Product	Original	13	128,065.60	0.2107
Sales	L7_L8_Excluded	15	153,546.41	0.1925
Sales	Original	15	153,546.41	0.1925

KEY INSIGHT

Product's Gini falls from 0.2107 to 0.1835 (−12.9%) and it drops from the most unequal department to the fourth. Operations barely moves (0.2057 to 0.2049) even though its salary variance falls by 26.9%, because removing the L8 also lowers the mean (178,214.97 to 169,889.99) and the Gini is scale-free: the spread shrinks in proportion to the mean. Product's inequality ranking therefore depends on one L7 employee (EMP0300).

Step 4 — Ranking single-employee removals by variance change

For every employee, the department's population variance of base salary is recomputed without that person (a correlated subquery) and compared with the original. A department must retain at least five people after the removal (`original_count - 1 >= 5`).

QUESTION 3, STEP 4 — INDIVIDUAL REMOVAL RANKING

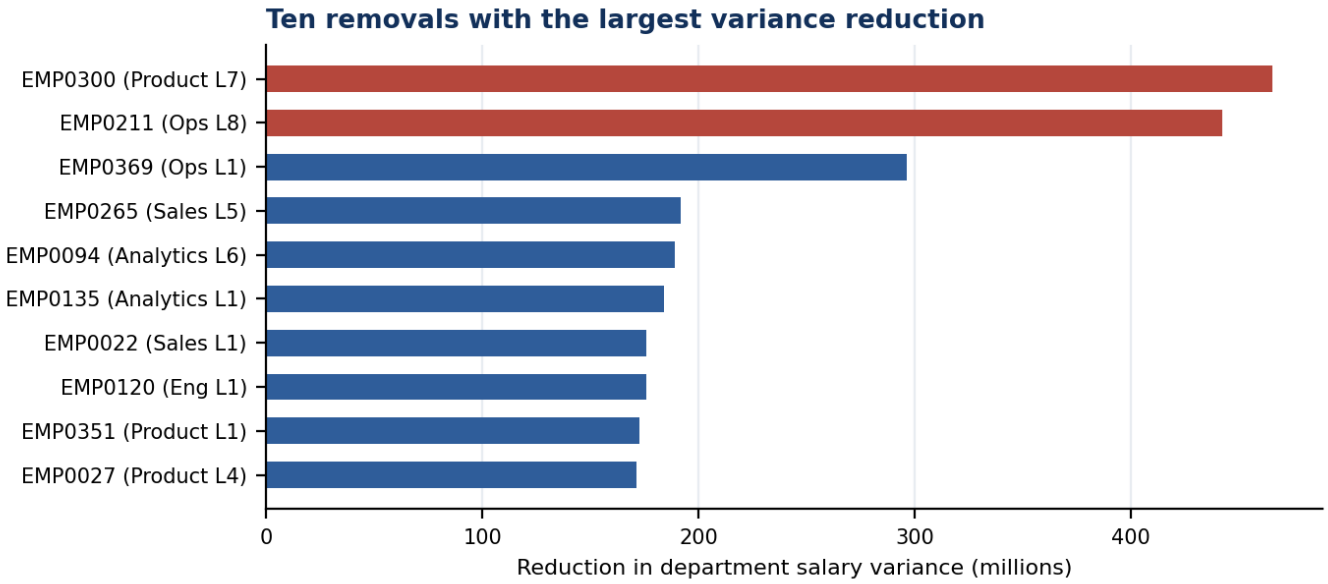
```
WITH department_sizes AS (
  SELECT
    department,
    COUNT(*) AS original_count
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
  GROUP BY department),
original_variance AS (
  SELECT
    department,
    COUNT(*) AS employee_count,
    VAR_POP(base_salary) AS original_variance
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
  GROUP BY department
),
removal_candidates AS (
  SELECT
    e.employee_id,
    e.department,
    e.job_level,
    e.base_salary,
    d.original_count
  FROM `employee_compensation (1)` e
  JOIN department_sizes d
    ON e.department = d.department
  WHERE e.currency = 'USD'
    AND d.original_count - 1 >= 5
),
variance_after_removal AS (
  SELECT
    c.employee_id,
    c.department,
    c.job_level,
    c.base_salary,
    o.original_variance,
    ( SELECT VAR_POP(e2.base_salary)
      FROM `employee_compensation (1)` e2
      WHERE e2.currency = 'USD'
        AND e2.department = c.department
        AND e2.employee_id <> c.employee_id
    ) AS revised_variance
  FROM removal_candidates c
  JOIN original_variance o
    ON c.department = o.department
```

QUESTION 3, STEP 4 — INDIVIDUAL REMOVAL RANKING

```
)
SELECT
  employee_id,
  department,
  job_level,
  base_salary,
  ROUND(original_variance, 2) AS original_variance,
  ROUND(revised_variance, 2) AS revised_variance,
  ROUND(revised_variance - original_variance, 2)
    AS variance_change
FROM variance_after_removal
ORDER BY variance_change ASC
LIMIT 20;
```

OUTPUT · REMOVALS WITH THE LARGEST VARIANCE REDUCTION (FIRST 12 OF 20)

employee_id	department	level	base_salary	original_variance	revised_variance	variance_change
EMP0300	Product	L7	174,000	1,333,147,929	867,576,388.9	−465,571,540.1
EMP0211	Operations	L8	195,000	1,644,925,620	1,202,610,000	−442,315,619.8
EMP0369	Operations	L1	56,000	1,644,925,620	1,348,560,000	−296,365,619.8
EMP0265	Sales	L5	175,000	1,285,582,222	1,093,882,653	−191,699,569.2
EMP0094	Analytics	L6	152,000	951,061,224.5	761,775,147.9	−189,286,076.6
EMP0135	Analytics	L1	40,000	951,061,224.5	767,076,923.1	−183,984,301.4
EMP0022	Sales	L1	55,000	1,285,582,222	1,109,801,020	−175,781,201.8
EMP0120	Engineering	L1	44,000	906,146,683.7	730,395,061.7	−175,751,622
EMP0351	Product	L1	38,000	1,333,147,929	1,160,354,167	−172,793,762.3
EMP0027	Product	L4	150,000	1,333,147,929	1,161,909,722	−171,238,206.8
EMP0069	Engineering	L6	189,000	906,146,683.7	745,113,854.6	−161,032,829.1
EMP0060	Analytics	L1	50,000	951,061,224.5	851,100,591.7	−99,960,632.77



KEY INSIGHT

The two largest variance reductions are exactly the L7 and L8 employees, so the removal ranking agrees with Step 1. From third place on, the biggest movers are not senior: seven of the top 20 are L1 employees paid 38,000–56,000, whose low pay also stretches the distribution. Variance is therefore reduced by removing *either* tail. The minimum-size rule never binds: the smallest department (Operations, 11 people) still has ten after one removal.

SECTION

07 Compensation Trajectory & Benchmark Instability

Question 4: three ranking definitions, rank movers and employees stable in the top 10%.

Business question. Calculate rankings under all employees, active employees only, and active employees hired within the last five years. Identify employees whose ranking improves by at least five positions when exited employees are excluded, and employees who remain in the top 10% under all three definitions.

Step 1 — Employment status in the cohort

QUESTION 4, STEP 1 — STATUS COUNTS

```
SELECT
  employment_status,
  COUNT(*) AS employee_count
FROM `employee_compensation (1)`
WHERE currency = 'USD'
GROUP BY employment_status;
```

OUTPUT · USD EMPLOYEES BY STATUS

employment_status	employee_count
Active	72
On Leave	3
Exited	6

The three definitions therefore cover 81 (all), 72 (Active only) and a smaller subset of Active employees hired in the last five years.

Step 2 — Rankings under the three definitions

Each definition has its own `RANK() OVER (ORDER BY total_compensation DESC)`. The results are joined back with LEFT JOINs, so an employee who is not part of a definition shows NULL for that rank.

QUESTION 4, STEP 2 — THREE RANKINGS SIDE BY SIDE

```
WITH employee_data AS (
  SELECT
    employee_id,
    employment_status,
    hire_date,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
all_rankings AS (
  SELECT
    *,
    RANK() OVER (
      ORDER BY total_compensation DESC
    ) AS all_employee_rank
  FROM employee_data
),
active_rankings AS (
  SELECT
    employee_id,
    RANK() OVER (
      ORDER BY total_compensation DESC
    ) AS active_employee_rank
  FROM employee_data
  WHERE employment_status = 'Active'
),
recent_active_rankings AS (
  SELECT
    employee_id,
    RANK() OVER (
      ORDER BY total_compensation DESC
    ) AS recent_active_rank
  FROM employee_data
```

QUESTION 4, STEP 2 — THREE RANKINGS SIDE BY SIDE

```
  WHERE employment_status = 'Active'
    AND hire_date >= DATE_SUB(CURDATE(), INTERVAL 5 YEAR)
)
SELECT
  a.employee_id,
  a.employment_status,
  a.hire_date,
  ROUND(a.total_compensation, 2) AS total_compensation,
  a.all_employee_rank,
  ar.active_employee_rank,
  rr.recent_active_rank
FROM all_rankings a
LEFT JOIN active_rankings ar
  ON a.employee_id = ar.employee_id
LEFT JOIN recent_active_rankings rr
  ON a.employee_id = rr.employee_id
ORDER BY a.all_employee_rank;
```

OUTPUT · TOP OF THE RANKING TABLE (FIRST 16 OF 29 ROWS SHOWN IN THE EXPORT)

employee_id	status	hire_date	total_comp	all_rank	active_rank	recent_active_rank
EMP0252	Active	10-02-2012	281,113.61	1	1	NULL
EMP0211	Active	19-06-2025	261,464.75	2	2	1
EMP0323	Active	06-11-2020	260,056.04	3	3	NULL
EMP0181	Active	30-07-2022	251,122.33	4	4	2
EMP0269	Active	28-04-2013	239,556.10	5	5	NULL
EMP0300	Active	03-04-2024	237,024.41	6	6	3
EMP0265	Active	09-07-2016	230,876.97	7	7	NULL
EMP0059	Active	23-11-2013	224,531.53	8	8	NULL
EMP0087	Active	07-03-2016	223,396.63	9	9	NULL
EMP0069	Active	14-03-2019	220,567.86	10	10	NULL
EMP0353	Active	05-02-2013	219,187.58	11	11	NULL
EMP0046	Active	23-03-2024	211,076.76	12	12	4
EMP0352	Active	04-10-2021	206,910.95	13	13	NULL
EMP0094	Active	11-06-2020	203,314.00	14	14	NULL
EMP0100	Active	07-08-2022	200,355.27	15	15	5
EMP0124	On Leave	14-08-2018	198,916.29	17	NULL	NULL

KEY INSIGHT

Only about one in three of the top-ranked employees belongs to the recent-hire cohort, and the recent cohort's leaders are not the overall leaders: EMP0211 (hired June 2025) is first among recent hires but second overall. EMP0124 (On Leave) is ranked 17th overall but has no active rank, which shifts everyone below it up by one in the active ranking (EMP0009 moves from 18th to 17th).

Step 3 — Employees whose rank improves by five or more positions

Original rank (all employees) is compared with the rank after removing Exited employees. Exited employees themselves are excluded from the result.

QUESTION 4, STEP 3 — RANK IMPROVEMENT OF 5 OR MORE

```
WITH employee_data AS (
  SELECT
    employee_id,
    employment_status,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
all_rankings AS (
  SELECT
    employee_id, employment_status,
    total_compensation,
```

QUESTION 4, STEP 3 — RANK IMPROVEMENT OF 5 OR MORE

```
    RANK() OVER (
      ORDER BY total_compensation DESC
    ) AS original_rank
  FROM employee_data
),
rankings_without_exited AS (
  SELECT
    employee_id,
    RANK() OVER (
      ORDER BY total_compensation DESC
    ) AS revised_rank
  FROM employee_data
  WHERE employment_status <> 'Exited'
)
SELECT
  a.employee_id,
  a.employment_status,
  ROUND(a.total_compensation, 2) AS total_compensation,
  a.original_rank,
  r.revised_rank,
  a.original_rank - r.revised_rank AS positions_improved
FROM all_rankings a
JOIN rankings_without_exited r
  ON a.employee_id = r.employee_id
WHERE a.employment_status <> 'Exited'
  AND a.original_rank - r.revised_rank >= 5
ORDER BY positions_improved DESC, a.employee_id;
```

OUTPUT · RANK MOVERS (13 EMPLOYEES)

employee_id	status	total_comp	original_rank	revised_rank	positions_improved
EMP0120	Active	57,422.30	80	74	6
EMP0351	Active	47,742.21	81	75	6
EMP0022	On Leave	72,554.23	78	73	5
EMP0029	Active	89,243.37	70	65	5
EMP0030	Active	75,888.60	77	72	5
EMP0053	Active	91,085.88	69	64	5
EMP0060	Active	83,212.64	73	68	5
EMP0071	Active	86,432.51	71	66	5
EMP0075	Active	81,193.40	74	69	5
EMP0104	Active	84,864.64	72	67	5
EMP0173	Active	76,307.64	75	70	5
EMP0204	Active	93,635.51	68	63	5
EMP0369	Active	75,929.62	76	71	5



KEY INSIGHT

The cohort has only six Exited employees, so the maximum possible improvement is six places; EMP0120 and EMP0351 reach it and the other eleven gain five. All thirteen sit between ranks 68 and 81, the lower end of the distribution. The effect is mechanical: it shows that at least five of the six exited employees were paid more than these people, not that their pay changed.

Step 4 — Employees who stay in the top 10% under all three definitions

Rankings use `ROW_NUMBER() … ORDER BY total_compensation DESC, employee_id` so ties cannot put two people on the same rank; the cut-off is `CEIL(count × 0.10)` within each definition.

QUESTION 4, STEP 4 — TOP 10% UNDER ALL THREE DEFINITIONS

```
WITH employee_data AS (
  SELECT
    employee_id,
    employment_status,
    hire_date,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
all_rankings AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      ORDER BY total_compensation DESC, employee_id
    ) AS all_rank,
    COUNT(*) OVER () AS all_count
  FROM employee_data),
active_rankings AS (
  SELECT
    employee_id,
    ROW_NUMBER() OVER (
      ORDER BY total_compensation DESC, employee_id
    ) AS active_rank,
    COUNT(*) OVER () AS active_count
  FROM employee_data
  WHERE employment_status = 'Active'
),
recent_active_rankings AS (
  SELECT
    employee_id,
    ROW_NUMBER() OVER (
      ORDER BY total_compensation DESC, employee_id
    ) AS recent_rank,
    COUNT(*) OVER () AS recent_count
  FROM employee_data
  WHERE employment_status = 'Active'
    AND hire_date >= DATE_SUB(CURDATE(), INTERVAL 5 YEAR)
)
SELECT
  a.employee_id,
  ROUND(a.total_compensation, 2) AS total_compensation,
  a.all_rank,
  ar.active_rank,
  rr.recent_rank
FROM all_rankings a
JOIN active_rankings ar
  ON a.employee_id = ar.employee_id
JOIN recent_active_rankings rr
  ON a.employee_id = rr.employee_id
WHERE a.all_rank <= CEIL(a.all_count * 0.10)
  AND ar.active_rank <= CEIL(ar.active_count * 0.10)
  AND rr.recent_rank <= CEIL(rr.recent_count * 0.10)
ORDER BY a.total_compensation DESC;
```

OUTPUT · EMPLOYEES IN THE TOP 10% UNDER ALL THREE DEFINITIONS

employee_id	total_compensation	all_rank	active_rank	recent_rank
EMP0211	261,464.75	2	2	1
EMP0181	251,122.33	4	4	2

KEY INSIGHT

Only two employees are stable: EMP0211 and EMP0181. Both were hired inside the five-year window (June 2025 and July 2022) and rank near the top of every list. EMP0252, EMP0323 and EMP0269 (overall ranks 1, 3 and 5) fall out of the third definition simply because they were hired before the window, and EMP0300 (recent rank 3) misses the recent-cohort cut-off, so “top 10%” depends strongly on which comparison group is used.

SECTION

08 Executive Recommendation

Question 5: the five ranked compensation-architecture risks with indicator, population and magnitude.

Business question. Produce a ranked list of five compensation-architecture risks, each supported by a measurable indicator, affected population and magnitude of discrepancy.

The original script contains four measurement queries for this question (Steps 1–4) but no final query or table that ranks the risks. Steps 1–4 are reproduced below; the ranked list that follows them was compiled from the results of Sections 04–07 and is part of this edition rather than of the original script.

Step 1 — Inequality by department

QUESTION 5, STEP 1 — DEPARTMENT GINI AND MEAN

```
WITH compensation_data AS (
  SELECT
    employee_id,
    department,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
department_totals AS (
  SELECT department,
    COUNT(*) AS employee_count,
    SUM(total_compensation) AS total_compensation,
    AVG(total_compensation) AS mean_compensation
  FROM compensation_data
  GROUP BY department
),
pairwise_differences AS (
  SELECT
    a.department,
    SUM(ABS(a.total_compensation - b.total_compensation))
      AS total_absolute_difference
  FROM compensation_data a
  JOIN compensation_data b
    ON a.department = b.department
  GROUP BY a.department
)
SELECT
  d.department,
  d.employee_count,
  ROUND(d.mean_compensation, 2) AS mean_compensation,
  ROUND(
    p.total_absolute_difference
      / (2 * d.employee_count * d.total_compensation),
    4
  ) AS gini_coefficient
FROM department_totals d
JOIN pairwise_differences p
  ON d.department = p.department
ORDER BY gini_coefficient DESC;
```

OUTPUT · DEPARTMENT INEQUALITY

department	employee_count	mean_compensation	gini_coefficient
Product	13	128,065.60	0.2107
Operations	11	178,214.97	0.2057
Sales	15	153,546.41	0.1925
Analytics	14	144,227.64	0.1845
Engineering	28	165,053.16	0.1465

Step 2 — Pay inequity risk (high performers below median)

QUESTION 5, STEP 2 — ABOVE-AVERAGE RATING, BELOW-MEDIAN PAY

```
WITH employee_data AS (
  SELECT
    employee_id,
    department,
    performance_rating,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
),
department_medians AS (
  SELECT
    department,
    AVG(total_compensation) AS median_compensation
  FROM (
    SELECT
      department,
      total_compensation,
      ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY total_compensation
      ) AS rn, COUNT(*) OVER (
        PARTITION BY department
      ) AS cnt
    FROM employee_data
  ) ranked
  WHERE rn IN (
    FLOOR((cnt + 1) / 2),
    FLOOR((cnt + 2) / 2)
  )
  GROUP BY department
),
performance_medians AS (
  SELECT
    department,
    AVG(performance_rating) AS median_performance
  FROM employee_data
  GROUP BY department
)
SELECT
  e.employee_id,
  e.department,
  e.performance_rating,
  ROUND(e.total_compensation, 2) AS total_compensation,
  ROUND(d.median_compensation, 2) AS department_median,
  ROUND(
    d.median_compensation - e.total_compensation, 2
  ) AS compensation_gap
FROM employee_data e
JOIN department_medians d
  ON e.department = d.department
JOIN performance_medians p
  ON e.department = p.department
WHERE e.total_compensation < d.median_compensation
  AND e.performance_rating > p.median_performance
ORDER BY compensation_gap DESC;
```

OUTPUT · EMPLOYEES BELOW MEDIAN PAY WITH AN ABOVE-AVERAGE RATING (FIRST 12 OF 26)

employee_id	department	rating	total_comp	department_median	compensation_gap
EMP0120	Engineering	4	57,422.30	164,020.73	106,598.43
EMP0030	Engineering	5	75,888.60	164,020.73	88,132.13
EMP0173	Analytics	4	76,307.64	151,338.91	75,031.27
EMP0029	Engineering	5	89,243.37	164,020.73	74,777.36
EMP0351	Product	3	47,742.21	119,418.25	71,676.04
EMP0071	Sales	4	86,432.51	150,499.50	64,066.99
EMP0053	Sales	4	91,085.88	150,499.50	59,413.62
EMP0078	Analytics	5	99,553.46	151,338.91	51,785.45
EMP0006	Sales	5	100,903.77	150,499.50	49,595.73
EMP0202	Operations	4	135,591.44	175,642.30	40,050.86
EMP0037	Operations	4	136,141.89	175,642.30	39,500.41
EMP0075	Product	3	81,193.40	119,418.25	38,224.85

NOTE ON THIS RESULT

This query returns 26 employees while Question 1.2 returns 7, because here the “performance median” is calculated with AVG (the department’s *mean* rating), so EMP0351 with a rating of 3 qualifies in Product. The seven employees of Question 1.2 are all inside these 26. See Defect 1 in Section 09. The ranked list below uses the stricter median-based seven.

Step 3 — Hiring-sequence inversions by department and job family

QUESTION 5, STEP 3 — INVERSION COUNT AND GAP

```
WITH employee_pairs AS (
  SELECT
    earlier.employee_id AS earlier_employee,
    later.employee_id AS later_employee,
    earlier.department,
    earlier.job_family,
    earlier.hire_date AS earlier_hire_date, later.hire_date AS later_hire_date,
    earlier.base_salary AS earlier_salary,
    later.base_salary AS later_salary,
    later.base_salary - earlier.base_salary AS salary_gap
  FROM `employee_compensation (1)` earlier
  JOIN `employee_compensation (1)` later
    ON earlier.department = later.department
   AND earlier.job_family = later.job_family
   AND earlier.hire_date < later.hire_date
  WHERE earlier.currency = 'USD'
    AND later.currency = 'USD'
    AND later.base_salary > earlier.base_salary
)
SELECT
  department,
  job_family,
  COUNT(*) AS inversion_count,
  ROUND(MAX(salary_gap), 2) AS largest_salary_gap,
  ROUND(AVG(salary_gap), 2) AS average_salary_gap
FROM employee_pairs
GROUP BY department, job_family
ORDER BY inversion_count DESC, largest_salary_gap DESC;
```

OUTPUT · INVERSION COUNTS BY DEPARTMENT AND JOB FAMILY (FIRST 10 OF 18 ROWS)

department	job_family	inversion_count	largest_salary_gap	average_salary_gap
Engineering	Data	7	129,000	62,571.43
Product	Product	6	105,000	57,333.33
Engineering	People	5	50,000	30,000
Analytics	Sales	3	56,000	37,333.33
Sales	Finance	2	91,000	74,500
Analytics	People	2	91,000	74,500
Product	Operations	2	60,000	40,500
Sales	Product	1	109,000	109,000
Operations	Finance	1	95,000	95,000
Product	Finance	1	61,000	61,000

The 18 groups sum to 38 inversions, matching Question 2. The largest single base-salary gap between an earlier and a later hire is 129,000 (Engineering, Data).

Step 4 — Compensation gaps among comparable employees

QUESTION 5, STEP 4 — PEER-CELL MIN, MAX AND GAP

```
SELECT
  department,
  job_level,
  job_family,
  work_location,
  COUNT(*) AS employee_count,
  ROUND(MIN(total_compensation), 2) AS minimum_compensation,
  ROUND(MAX(total_compensation), 2) AS maximum_compensation,
  ROUND(
    MAX(total_compensation) - MIN(total_compensation), 2
  ) AS compensation_gap
FROM (
  SELECT
    employee_id,
    department,
    job_level,
    job_family,
    work_location,
    base_salary
      + COALESCE(annual_bonus, 0)
      + COALESCE(equity_value, 0) AS total_compensation
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
) AS employee_data
GROUP BY
  department,
  job_level,
  job_family,
  work_location
HAVING COUNT(*) >= 2
ORDER BY compensation_gap DESC;
```

OUTPUT · PEER CELLS WITH A GAP

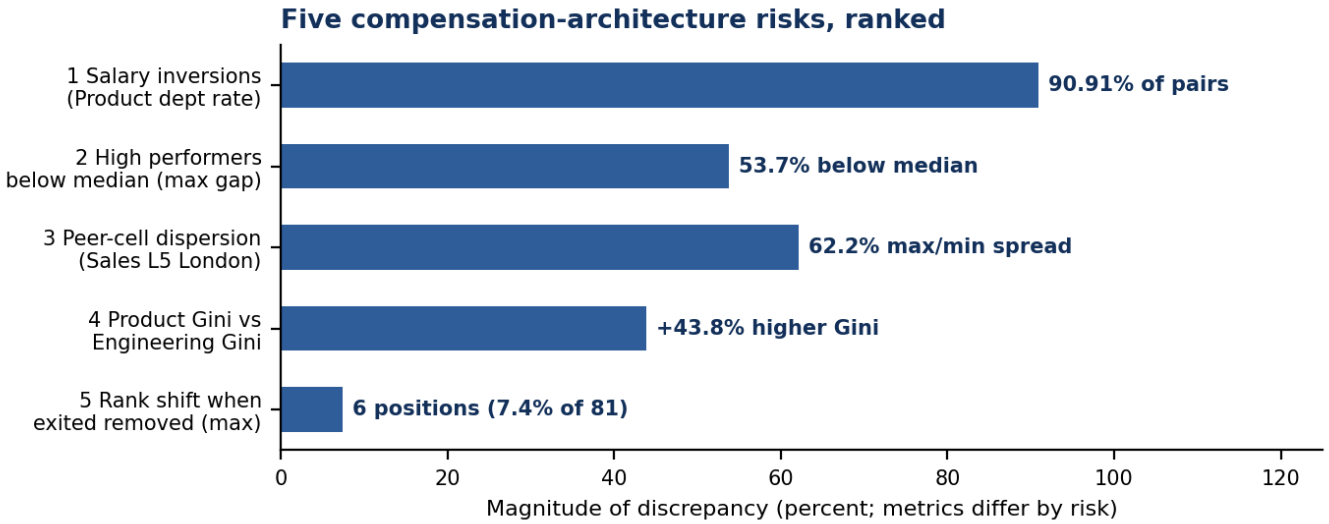
department	level	family	location	employee_ count	minimum	maximum	compensation_ gap
Sales	L5	Operations	London	2	147,731.69	239,556.10	91,824.41
Analytics	L4	Operations	New York	2	135,592.42	147,284.81	11,692.39
Engineering	L5	Operations	Bengaluru	2	156,984.57	166,036.10	9,051.53

The five ranked risks

Risks are ranked by a combination of the share of the relevant population affected and the size of the discrepancy. The ranking is a judgement built on the figures above; the figures themselves are exact outputs of the queries.

OUTPUT · RANKED COMPENSATION-ARCHITECTURE RISKS

#	Risk	Measurable indicator	Affected population	Magnitude of discrepancy
1	**Hiring-sequence salary inversions**	Inversion rate = inverted ÷ comparable pairs (Q2, Q5 step 3)	88 pairs, 38 inverted, in 5 departments; Product 10 of 11	43.18% overall; Product 90.91%; largest base gap 129,000 (Engineering, Data)
2	**High performers paid below median**	Rating above department median and pay below department median (Q1.2)	7 employees (8.6%): 4 Engineering, 1 each Analytics, Product, Sales	Up to 88,132.13 below median (53.7%, EMP0030); 26 employees on the broader mean-rating test
3	**Unexplained dispersion inside identical peer cells**	Max − min total compensation in cells of same department, level, family and location (Q1.4)	6 employees in 3 cells (7.4% of cohort)	Sales L5 Operations London: 91,824.41 (62.2% above the lower earner); other cells 8.6% and 5.8%
4	**Inequality concentrated in Product and Operations, driven by one L7/L8 each**	Department Gini; change in Gini and variance when L7/L8 excluded (Q1.1, Q3)	Product 13 employees; Operations 11; two individuals (EMP0300, EMP0211)	Product Gini 0.2107 vs Engineering 0.1465 (+43.8%); excluding L7 cuts Product variance by 34.92% and Gini by 12.9%
5	**Unstable benchmarks: ranks depend on the comparison group**	Rank change when exited staff are removed; membership of top 10% across three definitions (Q4)	13 employees move up 5–6 places (16.0%); 2 of 9 top-decile employees stay top under all three	Maximum shift 6 places; 7 of the 9 overall top-decile earners are not top 10% under the recent-hire definition



EXECUTIVE SUMMARY

The dominant risk is not the level of pay but its ordering: in the USD cohort, 43 of every 100 pairs of same-family colleagues have the newer hire paid more, and in Product it is 91 of 100. Combined with seven strongly rated employees paid well below their department median, the evidence is consistent with starting salaries being set without reference to existing staff and with little in-role adjustment, although the data cannot confirm the cause. Inequality (risk 4) is real but moderate, and rests on two senior individuals; benchmark instability (risk 5) is a measurement caveat to fix before any pay action is based on rankings.

LIMITS OF THE EVIDENCE

Every figure describes 81 employees paid in USD; peer groups are very small (two people in each Question 1.4 cell); and Question 2 pairs are not restricted to the same job level (Defect 5). The ranking of risks should be re-checked after the corrections in Section 09 are applied to the full dataset.

SECTION

09 Code Review & Corrections

Defects found while checking the script against its own requirements, with the fix for each.

The script was reviewed against the business questions printed in its own comments. Seven defects and seven design observations were found. The corrections below are provided as SQL; **the source CSV was not part of this edition, so corrected queries are shown but were not re-executed**, and the result tables elsewhere in this document are those of the original script.

Defects

Defect 1 — “median_performance” is a mean

SEVERITY: LOGIC · STATUS: CORRECTION PROVIDED

In Question 5, step 2 the CTE named `performance_medians` computes `AVG(performance_rating)`, while Question 1.2 uses a true median. The two queries answer the same business question but return 26 and 7 employees. Because a mean can fall between integer ratings, an employee whose rating equals the median (for example a 3 in Product) can pass the above-the-mean test while failing the median test.

CORRECTION — DEFECT 1

```
-- before (Q5 step 2)
performance_medians AS (
  SELECT department, AVG(performance_rating) AS median_performance
  FROM employee_data
  GROUP BY department
)
-- after: reuse the ROW_NUMBER median exactly as in Question 1.2
performance_medians AS (
  SELECT department, AVG(performance_rating) AS median_performance
  FROM (
    SELECT department, performance_rating,
           ROW_NUMBER() OVER (PARTITION BY department ORDER BY performance_rating) AS rn,
           COUNT(*)     OVER (PARTITION BY department) AS cnt
    FROM employee_data
  ) r
  WHERE rn IN (FLOOR((cnt + 1) / 2), CEIL((cnt + 1) / 2))
  GROUP BY department
)
```

Defect 2 — The five-year window moves every day

SEVERITY: LOGIC · STATUS: CORRECTION PROVIDED

Questions 4 (steps 2 and 4) filter `hire_date >= DATE_SUB(CURDATE(), INTERVAL 5 YEAR)`. CURDATE() changes daily, so the recent-hire cohort, its ranks and the “stable top 10%” list change on each run and cannot be reproduced or audited later.

CORRECTION — DEFECT 2

```
-- fix the reporting date once and reference it
WITH report_period AS (
  SELECT DATE '2026-10-27' AS as_of_date            -- set explicitly
),
recent_active_rankings AS (
  SELECT e.employee_id,
         RANK() OVER (ORDER BY e.total_compensation DESC) AS recent_active_rank
  FROM employee_data e
  CROSS JOIN report_period rp
  WHERE e.employment_status = 'Active'
    AND e.hire_date >= DATE_SUB(rp.as_of_date, INTERVAL 5 YEAR)
)
```

Defect 3 — Two tie conventions for “rank”

SEVERITY: LOGIC · STATUS: FIX DESCRIBED

Question 4 steps 2 and 3 use `RANK()` (ties share a rank), while step 4 uses `ROW_NUMBER() … employee_id` (ties broken alphabetically). The rank columns in step 4 can therefore disagree with step 2 for tied employees, and the top-10% cut-off `CEIL(count × 0.10)` is applied to arbitrary tie-break positions. Choose one convention (RANK for reporting, with ties kept together) and use it in all three steps.

Defect 4 — Question 1 is only partly answered, and “Task 3” is missing

SEVERITY: COMPLETENESS · STATUS: CORRECTION PROVIDED

The tasks are labelled Task 1, Task 2, Task 4 — there is no Task 3 — and the last requirement (“whether disparities persist among comparable job levels, job families and locations”) is answered only by one exact-match cell on all four attributes, which yields three cells of two people. It does not show whether a disparity persists along each dimension. The query below measures the spread of total compensation separately by job level, job family and location.

CORRECTION — DEFECT 4 (NEW QUERY)

```
WITH comp AS (
  SELECT job_level, job_family, work_location, department,
         base_salary + COALESCE(annual_bonus, 0) + COALESCE(equity_value, 0) AS total_comp
  FROM `employee_compensation (1)`
  WHERE currency = 'USD'
)
SELECT 'job_level' AS dimension, job_level AS segment, COUNT(*) AS n,
       ROUND(AVG(total_comp), 2) AS mean_comp, ROUND(STDDEV_POP(total_comp) / AVG(total_comp), 4) AS coeff_variation
FROM comp GROUP BY job_level
UNION ALL
SELECT 'job_family', job_family, COUNT(*), ROUND(AVG(total_comp), 2), ROUND(STDDEV_POP(total_comp) / AVG(total_comp), 4)
FROM comp GROUP BY job_family
UNION ALL
SELECT 'work_location', work_location, COUNT(*), ROUND(AVG(total_comp), 2), ROUND(STDDEV_POP(total_comp) / AVG(total_comp), 4)
FROM comp GROUP BY work_location
ORDER BY dimension, coeff_variation DESC;
```

Defect 5 — Inversion pairs ignore job level

SEVERITY: LOGIC · STATUS: CORRECTION PROVIDED

Question 2 pairs every earlier hire with every later hire in the same department and job family, regardless of level. A later L6 earning more than an earlier L3 is a promotion effect, not an inversion, yet it is counted (the level table shows L3→L6, L2→L7 and L3→L8 pairs at 100%). The 43.18% overall and 90.91% Product rates are therefore upper bounds. Add the level to the join to measure like-for-like inversions, then re-run Steps 1–5.

CORRECTION — DEFECT 5

```
-- before
JOIN `employee_compensation (1)` e2
  ON e2.department = e1.department
 AND e2.job_family = e1.job_family
 AND e2.currency   = e1.currency
 AND e2.hire_date  > e1.hire_date
-- after: compare people at the same level only
JOIN `employee_compensation (1)` e2
  ON e2.department = e1.department
 AND e2.job_family = e1.job_family
 AND e2.job_level  = e1.job_level
 AND e2.currency   = e1.currency
 AND e2.hire_date  > e1.hire_date
```

Defect 6 — Only 20.25% of employees are analysed

SEVERITY: DATA SCOPE · STATUS: DOCUMENTED

All queries filter `currency = 'USD'`, excluding 319 of 400 employees (INR 212, SGD 64, GBP 43). Pay is stored in local currency with no exchange-rate table, and currency does not follow work location (for example EMP0003, London, paid in INR; EMP0004, Mumbai, paid in GBP), so the exclusion cannot be explained by geography. Findings cannot be generalised to the company. Fix: add a `fx_rates(currency, rate_to_usd, as_of_date)` table, convert, and compute within-currency comparisons as a cross-check.

Defect 7 — hire_date may be stored as text

SEVERITY: VERIFY · STATUS: OPEN

The export shows dates as dd-mm-yyyy. If the import wizard created `hire_date` as VARCHAR, then `e2.hire_date > e1.hire_date` compares strings ("05-11-2021" sorts after "01-01-2025") and DATE_SUB on text is unreliable. Check with SHOW COLUMNS and convert if needed. The pair-ordering and five-year-window results should be treated as unverified until this is confirmed.

CORRECTION — DEFECT 7

```
SHOW COLUMNS FROM `employee_compensation (1)` LIKE 'hire_date';
-- if the type is varchar/text:
ALTER TABLE `employee_compensation (1)` ADD COLUMN hire_dt DATE;
UPDATE `employee_compensation (1)`
SET hire_dt = STR_TO_DATE(hire_date, '%d-%m-%Y');
-- then use hire_dt in every comparison
```

Design observations

Observation 8 — Table name

The wizard-generated name `employee_compensation (1)` needs backticks in every query. Rename once: `RENAME TABLE `employee_compensation (1)` TO employee_compensation;`.

Observation 9 — Script hygiene

The Step 4 import is an empty comment, so the script is not re-runnable; add a CREATE TABLE and LOAD DATA step (or document the wizard). A heading reads "Total Employees aur Currency Check" — mixed-language text in comments should be standardised.

Observation 10 — Dead code in Question 3, step 2

The columns `revised_rank` and `revised_mean_check` (and the CTE that carries them) are never used; the revised median is computed separately in `revised_ranked`. Remove them.

Observation 11 — The minimum-size rule never binds

`original_count - 1 >= 5` passes for all departments because the smallest USD department has 11 people. A threshold that excludes nothing gives no protection; set it from the question (for example 10 after removal) or report why none is needed.

Observation 12 — Mixed measurement bases

Question 3 compares mean, median and variance of *base salary* but Gini of *total compensation*, so the two halves of the answer are not on the same footing. State this in the output or add total-compensation variants.

Observation 13 — Tiny segments

Question 2 steps 3–5 return mostly cells of one or two pairs at 100%. Add `HAVING COUNT(*) >= 5` (or report a confidence caveat) so that the ordering by rate is not driven by single pairs.

Observation 14 — Missing constraints

The source table has no primary key, no CHECK on `performance_rating BETWEEN 1 AND 5`, and no foreign key for `manager_id`. A key on employee_id would formalise what Step 4 verified.

SECTION

10 Results, Setup & Roadmap

How to run the project, object inventory and future enhancements.

How to Run

- Install MySQL 8.0 or later (window functions, CTEs and VAR_POP are required) and open MySQL Workbench.
- Run Step 1 of Section 03 to create and select `compensation_analysis`.
- Import the employee-compensation CSV with the Table Data Import Wizard; rename the table (Observation 8) or keep the backticked name.
- Confirm the 400 rows and the currency counts of Section 03, Step 4; confirm the hire_date column type (Defect 7).
- Run Sections 04 to 08 in order; each query is independent and read-only.
- Apply the corrections of Section 09 and re-run Questions 2 and 4 before acting on the inversion and ranking results.

Object Inventory

OUTPUT · WHAT THE PROJECT CONTAINS

Object	Count	Detail
Database	1	compensation_analysis
Source table	1	employee_compensation (1) — 13 analytical columns, 400 rows
Cohort analysed	81 of 400	USD-paid employees (20.25%)
Analytical questions	5	Inequality; inversions; counterfactuals; benchmark instability; executive summary
Queries	24	3 setup/validation, 4 Question 1, 5 Question 2, 4 Question 3, 4 Question 4, 4 Question 5
Charts in this document	7	Currency mix, Gini, inversion rate, variance, removal ranking, rank movers, ranked risks
Defects / observations	7 / 7	Section 09

Headline Results

OUTPUT · SUMMARY OF FINDINGS (USD COHORT, 81 EMPLOYEES)

Question	Finding
Inequality	Gini from 0.1465 (Engineering) to 0.2107 (Product); Operations median highest (175,642.30), Product lowest (119,418.25)
High performers under median	7 employees by the strict test, up to 88,132.13 (53.7%) below median; 26 by the mean-rating test
Top-decile anomaly	1 of 9 (EMP0323), 3.9% below a two-person peer median
Inversions	38 of 88 pairs (43.18%); Product 90.91%; EMP0030 under-paid relative to four later hires
Counterfactual	Excluding L7/L8 changes only Operations and Product; Product variance −34.92%, Gini 0.2107 → 0.1835
Rank stability	13 employees gain 5–6 places when exited staff are removed; 2 employees (EMP0211, EMP0181) stay in the top 10% under all three definitions

Roadmap

- **Currency normalisation.** Add an exchange-rate table and widen the analysis from 81 to 400 employees.
- **Like-for-like inversions.** Apply Defect 5, add level-aware thresholds and a minimum pair count.
- **Statistical rigour.** Replace descriptive cells with regression (pay on level, location, tenure, rating) and report residuals instead of raw gaps.
- **Reproducibility.** Fix the reporting date (Defect 2) and parameterise the L7/L8 and top-N thresholds in a configuration CTE.
- **Schema.** Normalise into employees, departments, job_levels and pay_components tables with keys and CHECK constraints, and add a view for total compensation.

- **Dashboard.** Publish the Gini, inversion rate and rank-mover tables to a BI tool with a department and level filter.